PHASE 1 · FOUNDATIONS

Day 03 — Keys, signatures & transactions

Learning Blockchain in 15 Days · Day 3 of 15 · Saturday 17 October 2026 · Manish Khadka

Day 02's chain let anyone write "Sita pays Ram 5 coins". Today only Sita can: every payment carries a digital signature that only her private key can make and everyone can check with her public key.

Learning objectives

1. Explain the chain **private key → public key → address**, and why it only works in one direction.
2. Explain what an ECDSA signature (r, s) proves, and what it does *not* prove on its own.
3. List the fields a transaction must sign (domain, chain id, from, to, amount, nonce) and the attack each field blocks.
4. Recognise the classic signature bugs: replay, cross-chain replay, malleability (high-S), time-of-check/time-of-use, message-type confusion.
5. Use checksummed addresses to catch typos before coins are lost.

The concept, step by step

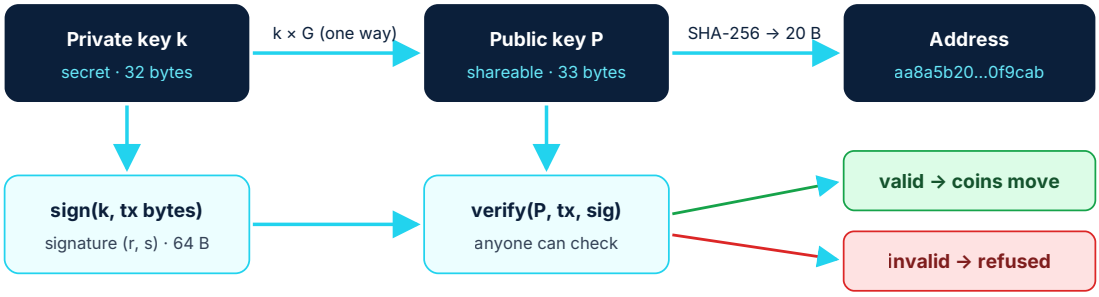


Figure 1 — The whole day in one picture. Only the holder of k can make a signature that P accepts.

Step 1: a private key is a secret number

On **secp256k1**, the curve Bitcoin and Ethereum use, a private key is any whole number from 1 to $n - 1$, with $n \approx 1.16 \times 10^{77}$. Like a PIN, except nobody can guess it: with about 10^{77} possible keys, trying them one by one is hopeless for any computer that will ever exist.

Step 2: the public key is computed from it, one way

`P = k × G`: "add" the curve's starting point *G* to itself *k* times (with a fast double-and-add trick). Going forward takes microseconds; going back, finding *k* from *P*, is the **discrete logarithm problem**. Like mixing paint: easy to mix, impossible to un-mix.

Step 3: the address is a short fingerprint of the public key

This repo uses the first 20 bytes of SHA-256(public key). Bitcoin uses RIPEMD-160(SHA-256(key)); Ethereum the last 20 bytes of Keccak-256. Same idea: an account number you can share.

Step 4: a signature is a stamp tied to the secret *and* the exact message

The signer feeds the private key and the exact bytes into ECDSA and gets two numbers (r, s). Anyone with the public key can check them. Change one bit of the message, or use another key, and the check fails.

Step 5: a transaction is a signed message, and every field earns its place

A valid signature only proves "the owner of this key signed these bytes". It does not say *when*, *how many times* or *on which network*. That is the job of the other signed fields:

Field	Plain meaning	Attack it blocks
`domain`	"this is a transfer"	Reusing a note or login signature as a payment
`chainId`	which network	Cross-chain replay (testnet → mainnet), Ethereum's EIP-155
`from`	sender's public key	Signing "as Sita" with another key
`to`, `amount`	who gets how many whole coins	Editing or redirecting after signing; rounding tricks
`nonce`	"my payment number 0, 1, 2..."	Replay: the same payment applied twice

Step 6: one payment, one signature (low-S)

For any valid ECDSA signature (r, s), the pair (r, *n* − s) is also valid. Anyone can make this "twin" without the key, which changes anything that hashes the signature. Bitcoin (BIP 62/146) and Ethereum (EIP-2) force **s ≤ n/2**; this repo does the same on both sign and verify.

Key code walkthrough

The exact bytes that get signed — `src/transaction.js`

```
function signingBytes(tx) {
  assertBody(tx);  // version, chainId, key, 40-hex "to", safe whole amount ≥ 1, nonce ≥ 0
  return Buffer.from(JSON.stringify(
    ["learning-blockchain/transfer", tx.version, tx.chainId, tx.from, tx.to, tx.amount,
tx.nonce]));
}
// tx id = SHA-256(signingBytes): the signature is NOT part of the id
```

A fixed-order JSON *array* of already-validated primitives: no key order, no `toJSON`, no surprises (Day 02's lesson).

Low-S signing — `src/keys.js`

```
const sig = crypto.sign("sha256", messageBytes(message), { key, dsaEncoding: "ieee-p1363" });
let s = toBig(sig.subarray(32));
if (s > HALF_N) s = N - s;          // always the lower twin
// verify refuses r = 0, r ≥ n, s = 0 and s > n/2 before any maths
```

Read once, then check, then use — `verifyTransfer`

```
tx = snapshot(input);        // every field read exactly ONCE into a frozen copy (audit M1)
bytes = signingBytes(tx);
if (!verifySignature(tx.from, bytes, tx.signature)) return { ok: false, … };
return { ok: true, id, sender: addressFromPublicKey(tx.from), tx };  // callers use THIS tx
```

The ledger after the evening audit — `src/ledger.js`

```
class Ledger {
  #chainId; #totalSupply; #balances = new Map(); #nonces = new Map();   // private (E1)
  get chainId() { return this.#chainId; }                          // read-only
  applyAll(txs) {                                                    // all-or-nothing
    for (let i = 0; i < length; i++)
      if (!Object.hasOwn(txs, i)) throw new TypeError(`transaction ${i} is missing`); // E2
    … check every tx on a scratch copy, then commit once
  }
}
Object.freeze(Ledger.prototype);
```

Evening improvement: checksummed addresses

```
const h = sha256(address).toString("hex");       // hash of the lowercase address
for (let i = 0; i < 40; i++)
  out += parseInt(h[i], 16) >= 8 ? address[i].toUpperCase() : address[i];  // EIP-55 idea
```

Results from the run

```
$ npm start
Sita  address aa8a5b204a46…0f9cab
Ram   address 6e0a062d96a1…c7d57b
Hari  address 0007bbbef172…47087e
Sita → Ram 30, Ram → Hari 10
Sita 70 · Ram 20 · Hari 10
total 100 (supply 100)
```

```
$ npm run typo
AA8A5b204a46BE63Be0862C239aC0911E70f9CAB
typos tried:  10,000
caught:        9,999 (99.99%)
missed:            1

$ npm run bench   (2,000 payments)
sign 0.384 ms · verify 0.447 ms each
```

```
$ npm run attack    (13 tricks: forged key, edited amount, redirect, replay twice,
                    cross-chain replay, high-S twin, overdraft, 0.1 coins, 2^53+1 coins,
                    "__proto__" address, half-applied batch, garbage key, lying getter)
Mallory tried 13 attacks: 13 refused, 0 got through.

$ npm test           # tests 38 · pass 38 · fail 0
$ npm run lint       eslint: 0 problems
$ npm audit --omit=dev   found 0 vulnerabilities
```

Signatures and timings change per run (ECDSA uses a fresh random number each time; speed depends on the machine). Addresses are fixed because the demo keys are derived from names — never do that with real money.

QA & security findings

Morning (attack script + manual review): 10 fixed — **2 high** (B3 replay: Ram received 40 coins twice; M1 a getter read 1 coin when checked and 100 when paid), **6 medium** (high-S twin, fractions, half-applied batch, cross-chain replay, precision loss above 2^{53}, `__proto__` address), **2 low** (any string as recipient, crash on garbage key).

Evening (independent audit, failing test first):

ID	Severity	What was wrong	Fix
E1	**Medium**	Ledger state was public: `ledger.balances.set(mallory, 1e6)` minted a million coins; `ledger.chainId = "lb-testnet"` switched off the cross-chain check.	Private `#fields`, read-only getters, frozen instance, prototype and class. Only a verified transaction changes state.
E2	**Low**	`applyAll` skipped holes: a "3-transaction" sparse batch applied 1.	Copy the list once, refuse holes.
E3	**Low**	A `Map` as genesis gave a ledger with 0 coins; `null` crashed.	Genesis must be a plain object.
E4	**Low**	`verify` accepted `[360, 361]` as the bytes of "hi"; lone surrogates let one signature fit two strings; a number made "never throws" verify throw.	Only bytes or well-formed text; verify returns `false`.

Checked, not vulnerable: a compressed key with x ≥ p (an alias of a real point) is refused, so one key cannot have two addresses. **Still open (accepted):** random *k* instead of RFC 6979 (low); a wallet signs whatever bytes it is handed (info; real wallets show the user what they sign, EIP-712).

Common mistakes

- **"The signature is valid, so the payment is fine."** Without a nonce and chain id, a valid signature can be replayed forever.
- **Using the signature in the transaction id.** Malleable signatures then change the id (the Mt. Gox 2014 excuse).
- **Floating-point money.** 0.1 + 0.2 ≠ 0.3. Use whole units (satoshi, wei).
- **Checking one object, using another.** Read untrusted input once, freeze it, then verify and use that copy.
- **Public mutable state.** If code can write a balance directly, signatures protect nothing.
- **Reusing the random k.** Two signatures with the same k reveal the private key (Sony PS3, 2010).

5 interview questions

1. Why can't someone derive my private key from my public key?

Because that means solving the elliptic-curve discrete logarithm problem for a 256-bit group. The best known algorithms need about 2^{128} operations, far beyond any computer.

2. What is signature malleability and how do Bitcoin and Ethereum stop it?

If (r, s) is valid, so is (r, n − s). A third party can create the twin without the key. Both chains require the low-S form (BIP 146, EIP-2), and modern ids (SegWit txid) do not cover signatures.

3. What is a replay attack, and what are the two kinds you must block?

Re-submitting a valid signed transaction. Same chain: blocked by a per-account nonce (account model) or by spending each output once (UTXO). Another chain: blocked by signing the chain id (EIP-155).

4. Account model vs UTXO model?

Ethereum stores a balance and nonce per address; Bitcoin stores unspent outputs, each locked to a key, and a payment consumes outputs and creates new ones. UTXO is easier to verify in parallel; accounts are simpler for smart contracts.

5. What does an address checksum protect against, and what not?

It catches typos when a human copies an address (here 9,999 of 10,000 one-character typos). It does not protect against a correct-looking address swapped by malware (address poisoning).

Exercises (solutions tested in `exercises/solutions`)

Level	Exercise	Goal and hint
Easy	Sign a note, not a payment	`signNote` / `verifyNote` for "I, Sita, agree to meet Ram at 5 pm." Prove the note signature can never be used as a payment. *Hint: a different domain string.*
Medium	What goes into the next block?	`selectForBlock(ledger, pending)` for payments arriving out of order, duplicated, or with gaps. *Hint: only the next nonce per sender; repeat passes, because one payment can fund another.* Result: 3 selected, 1 dropped, 1 waiting.
Challenge	A 2-of-3 shared vault	Coins move only when 2 *different* owners sign the exact payment. Refuse a single owner, the same owner twice, outsiders, mismatched amounts and replays. *Hint: the address commits to the threshold and sorted keys.*

Cheat sheet

secp256k1	`y² = x³ + 7 (mod p)` ; group order n ≈ 1.16 × 10^{77}
Private → public	`P = k × G` (fast); P → k is the discrete log problem (infeasible)
Compressed key	33 bytes: `02` / `03` (y even/odd) + 32-byte x
Address	BTC: RIPEMD-160(SHA-256(P)) · ETH: last 20 B of Keccak-256(P) · here: first 20 B of SHA-256(P)
ECDSA signature	(r, s), 64 bytes; valid only for this key and these exact bytes
Low-S	s ≤ n/2 → exactly one valid form per signature
Replay protection	nonce (same chain) + chainId (other chains) + domain (other message types)
Money	whole units only (satoshi, wei); never floats
Checksum address	case of each letter = hash nibble ≥ 8 (EIP-55 idea)
Run	`npm start · npm run attack · npm run typo · npm test`

References

1. SEC 2: Recommended Elliptic Curve Domain Parameters (secp256k1, §2.4.1) — https://www.secg.org/sec2-v2.pdf
2. S. Nakamoto, Bitcoin: A Peer-to-Peer Electronic Cash System (§2 Transactions) — https://bitcoin.org/bitcoin.pdf
3. A. Antonopoulos, D. Harding, Mastering Bitcoin, 3rd ed., ch. 4 "Keys and Addresses" & ch. 8 "Digital Signatures" — https://github.com/bitcoinbook/bitcoinbook
4. A. Antonopoulos, G. Wood, Mastering Ethereum, ch. 4 "Cryptography" — https://github.com/ethereumbook/ethereumbook
5. ethereum.org, Ethereum accounts (keys, addresses, nonces) — https://ethereum.org/en/developers/docs/accounts/
6. EIP-155: Simple replay attack protection — https://eips.ethereum.org/EIPS/eip-155
7. EIP-55: Mixed-case checksum address encoding — https://eips.ethereum.org/EIPS/eip-55
8. BIP 146: Dealing with signature encoding malleability (low-S) — https://github.com/bitcoin/bips/blob/master/bip-0146.mediawiki
9. RFC 6979: Deterministic usage of DSA and ECDSA — https://www.rfc-editor.org/rfc/rfc6979
10. SWC-121: Missing protection against signature replay attacks — https://swcregistry.io/docs/SWC-121